Calculator Unit Testing - Implementation Document

Introduction

This exercise demonstrates how to write unit test cases for a Calculator application using the NUnit testing framework. Different types of testing such as parameterized testing, exception handling, testing void methods, and mocking concepts are covered.

Objective

- Implement parameterized test cases using TestCase attributes.
- Test methods that return values using Assert.AreEqual.
- Test exception handling using try-catch and Assert.Fail.
- Test void methods such as AllClear().
- Understand why private methods are generally not tested directly.
- Learn the purpose of mocking and the Moq framework.

Task 1: Parameterized Test Cases

Step 1: Open the Calculator project.
Step 2: Create CalculatorTests.cs.
Step 3: Add multiple [TestCase] attributes for subtraction.
Step 4: Verify results using Assert.AreEqual().
Step 5: Repeat the same process for multiplication.
Step 6: Repeat the same process for division.
Step 7: For division by zero, use a try-catch block to catch ArgumentException.
Step 8: If the exception is not thrown, use Assert.Fail("Division by zero").

Task 2: Testing Void Method

Step 1: Create a test method named TestAddAndClear().
Step 2: Call the Add() method.
Step 3: Verify the result using Assert.AreEqual().
Step 4: Call the AllClear() method.
Step 5: Verify that GetResult returns 0 using Assert.AreEqual().

Private Methods

Private methods are implementation details of a class. They are tested indirectly through public methods. Testing private methods directly increases maintenance effort and tightly couples test cases with internal implementation.

Mocking Framework

Mocking is used to replace external dependencies such as databases, file systems, or web services with fake objects. The Moq framework helps developers isolate business logic and perform faster, reliable unit testing.

Result

All parameterized test cases executed successfully. Addition, subtraction, multiplication, division, exception handling, and AllClear() functionality were verified using NUnit.

Conclusion

The Calculator application was successfully tested using NUnit. Parameterized tests reduced duplicate code, Assert methods verified expected outputs, exception handling improved reliability, and mocking concepts were understood for future applications.

Output